

BE PROJECT SEMESTER VII REPORT
for
CHECKMATE: SMART PLAGIARISM & AI DETECTOR

Jovita Furtado
Suhani Salgaonkar
Shrain Dhavjekar
Sohail Agha

BACHELOR OF ENGINEERING
IN
COMPUTER ENGINEERING

OF GOA UNIVERSITY



2025-2026

COMPUTER ENGINEERING
PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722

**PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722**



BE PROJECT SEMESTER VII REPORT

for

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

**Jovita Furtado
Shrain Dhavjekar
Suhani Salgaonkar
Sohail Agha**

**202208341
202208371
202208325
202208354**

Submitted as a requirement for Semester VII Project examination.

2025-2026

Approved By:

.....
Ms. Ramita Karpe
Guide

.....
Dr. Supriya Patil
Head of the Department

.....
Dr. Mahesh Parappagoudar
Principal

Examined By:

Examiner 1:
(Name / Signature / Date)

Examiner 2:
(Name / Signature / Date)

CONTENTS

CHAPTER	Page No.
Acknowledgement.....	i
Abstract.....	ii
List of Figures.....	iii
List of Tables.....	iv
List of Symbols, Abbreviations or Nomenclature	v
1. INTRODUCTION	
1.1 Purpose.....	9
1.2 Project Scope.....	9
1.3 Literature Survey.....	10
2. OVERALL DESCRIPTION	
2.1 Project Perspective.....	11
2.2 Project Functions.....	11
2.3 User Classes and Characteristics.....	12
2.4 Operating Environment.....	12
3. REQUIREMENTS	
3.1 Functional Requirements.....	13
3.2 Non-Functional Requirements.....	13
3.3 Hardware Requirements.....	14
3.4 Software Requirements.....	14
4. SYSTEM DESIGN	
4.1 System Architecture.....	15
4.2 Data Flow Diagram (DFD).....	15
4.3 Module Description.....	16
4.4 AI Model Integration.....	16
5. IMPLEMENTATION AND TESTING	
5.1 Copied Content Detection Model.....	17
5.2 Paraphrase Detection Model.....	18
5.3 AI-Generated Content Detection.....	18
5.4 Originality Scoring Engine.....	18
5.5 Application Features Implemented.....	19
CONCLUSION	23
REFERENCES	24

ACKNOWLEDGEMENT

We express our sincere gratitude to Padre Conceição College of Engineering, Verna, for providing us with the opportunity, resources, and institutional support required to carry out our project titled “CheckMate – AI Powered Plagiarism & Idea Originality Checker.” The encouragement to work on practical, impactful, and academically relevant engineering solutions has played a significant role in shaping the direction and quality of our work.

We extend our heartfelt thanks to our Principal, Dr. Mahesh B. Parappagoudar, for fostering an academic environment that promotes innovation, technical exploration, and project-oriented learning. His vision for academic excellence and holistic student development has been a constant source of motivation throughout the course of this project.

We are deeply grateful to our Head of the Department of Computer Engineering, Dr. Supriya A. Patil, for her continuous guidance, encouragement, and departmental support. Her leadership and emphasis on technical rigor, ethical practices, and professional discipline have helped us maintain clarity, consistency, and purpose in our project development.

We sincerely thank our project guide, Ms. Ramita Karpe, for her invaluable guidance, patient supervision, and constructive suggestions at every stage of the project. Her technical expertise, attention to detail, and continuous feedback greatly contributed to the improvement, refinement, and successful completion of this project.

Finally, we also extend our gratitude to our Review Team Members for their thoughtful evaluations, critical observations, and valuable recommendations. Their insights enabled us to identify areas of improvement and significantly enhanced the overall structure, technical depth, and quality of the project.

ABSTRACT

CheckMate, presents a comprehensive, AI-driven plagiarism and idea originality detection system designed to address critical challenges in academic integrity, such as direct copying, advanced paraphrasing, and the increasing misuse of AI-generated content. CheckMate is developed using a full-stack web architecture comprising React.js (Frontend), Node.js (Backend), Python-based AI processing modules, and MongoDB for data storage.

The core innovation of CheckMate lies in the integration of multiple Natural Language Processing and Deep Learning models for comprehensive text analysis:

1. Copied Content Detection Model (TF-IDF with Cosine Similarity):
This model represents documents as weighted term vectors and computes similarity scores to detect directly copied content. It is optimized for fast comparison and serves as the baseline plagiarism detection mechanism, enabling accurate identification of high textual overlap across documents.
2. Paraphrase Detection Model (SBERT – Sentence-BERT):
Fine-tuned for semantic similarity analysis, this model captures contextual meaning beyond surface-level text matching and effectively identifies paraphrased content where wording and sentence structure are altered while preserving the original meaning.
3. AI-Generated Content Detection Model (Transformer-Based Classifier):
This model analyzes linguistic patterns, coherence, and probability distributions to distinguish between human-written and AI-generated text. It assigns an AI-likelihood score, helping faculty identify submissions influenced by generative AI tools.
4. Originality Scoring Engine:
Outputs from all detection modules are combined using a weighted scoring mechanism to generate a comprehensive originality score. This score provides an interpretable measure of document authenticity and highlights specific areas requiring revision.

The CheckMate platform includes role-based dashboards for students and faculty, detailed originality reports with highlighted similarities, and submission history tracking.

LIST OF FIGURES

Figure	Name	Page No.
Fig. 4.1	Technologies Used	15
Fig. 4.2	System Methodology (Flow Diagram)	16
Fig. 5.1	Copied Content Detection (TF-IDF Model)	17
Fig. 5.5.i	Student Dashboard Interface	19
Fig. 5.5.ii	Faculty Dashboard Interface	20

LIST OF TABLES

Table No.	Name	Page No.
Table 2.1	CheckMate System Scope and Benefits	11
Table 2.2	User Classes and Characteristics	12
Table 3.1	Operating Environment Details	13
Table 3.2	Non-Functional Requirements	14
Table 3.3	Hardware Requirements	14
Table 3.4	Software Requirements	14
Table 4.1	Technology Integration Details	15
Table 4.2	AI & NLP Model Integration Details	16
Table 5.4	Interpretation of Similarity Detection Outcomes	19

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
DB	Database
DL	Deep Learning
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
LMS	Learning Management System
ML	Machine Learning
NLP	Natural Language Processing
PDF	Portable Document Format
REST	Representational State Transfer
SBERT	Sentence-BERT
TF-IDF	Term Frequency–Inverse Document Frequency
UI/UX	User Interface / User Experience
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

Academic integrity is a fundamental principle of education and research. However, the widespread availability of online resources and AI-based content generation tools has significantly increased the risk of plagiarism in academic submissions. Manual plagiarism checking is time-consuming and prone to human error, making automated systems essential. CheckMate aims to provide an efficient, accurate, and scalable solution for plagiarism and AI-content detection in academic environments.

1.1 Purpose

The purpose of this project is to design and implement an end-to-end intelligent system named CheckMate, aimed at ensuring academic originality and integrity in educational institutions. With the rapid increase in digital content availability and the widespread use of generative Artificial Intelligence tools, traditional plagiarism detection methods have become insufficient. Many existing systems fail to detect paraphrased or AI-generated content accurately, leading to unfair academic practices.

CheckMate aims to assist both students and faculty by providing a reliable, automated mechanism for detecting copied, paraphrased, and AI-generated text. The system focuses on transparency by highlighting problematic sections and explaining the nature of similarity detected. It also provides actionable feedback that helps students improve the originality of their work rather than merely penalizing them.

By integrating multiple Natural Language Processing (NLP) techniques, the project seeks to deliver higher accuracy and fairness in plagiarism detection. Ultimately, the goal is to promote ethical academic behavior, reduce faculty workload, and enhance the quality of academic submissions.

1.2 Project Scope

The scope of the CheckMate project includes the development of a web-based plagiarism and originality detection platform suitable for academic use. The system is designed to analyze assignments, project reports, and research papers submitted by students and faculty. It supports multiple document formats such as PDF, DOCX, and TXT, making it flexible and easy to integrate into existing academic workflows.

The platform covers multiple dimensions of originality detection, including direct copying, paraphrasing, and AI-generated content. It provides role-based access for students and faculty,

ensuring appropriate functionality and data visibility for each user group. Faculty members can manage assignments, set similarity thresholds, and review submissions, while students can upload documents and view detailed reports.

The project assumes institutional deployment and stable internet connectivity. It does not currently address multimedia plagiarism such as images, audio, or video content. However, the system architecture is designed to be scalable, allowing future enhancements and integration with institutional Learning Management Systems (LMS).

1.3 Literature Survey

The literature survey reveals that traditional plagiarism detection systems primarily rely on string matching and keyword-based techniques. While these methods are effective for detecting exact copies, they fail when the text is paraphrased or semantically altered. Researchers have highlighted the importance of semantic similarity detection using vector-based representations and contextual embeddings.

Recent studies emphasize the use of TF-IDF combined with cosine similarity as a baseline approach for identifying copied text. However, advancements in NLP have led to the adoption of transformer-based models such as BERT and SBERT, which capture contextual meaning and sentence-level similarity more effectively. These models significantly improve paraphrase detection accuracy.

Additionally, the emergence of AI-generated content has introduced new challenges. Research indicates that perplexity, burstiness, and transformer-based classifiers can be used to differentiate between human-written and AI-generated text. Existing commercial tools are often expensive and lack transparency, motivating the development of an affordable, explainable, and academically focused solution like CheckMate.

CHAPTER 2

OVERALL DESCRIPTION

2.1 Project Perspective

CheckMate is designed as a **standalone, full-stack web application** that replaces manual and semi-automated plagiarism checking methods currently used in many institutions. The system introduces a proactive, AI-driven approach to originality verification, shifting the focus from punishment to improvement and learning.

The platform acts as a centralized solution for students, faculty, and administrators. It integrates document processing, similarity analysis, AI detection, and reporting within a single system. The modular design ensures that each component operates independently while contributing to a unified originality score.

From an institutional perspective, CheckMate enhances academic governance by providing consistent and objective evaluation criteria. The system can be deployed at the department, college, or university level, making it suitable for large-scale academic environments.

Aspect	Description
Project Name	CheckMate – AI Powered Plagiarism & Idea Originality Checker
Application Type	Web-based Academic Integrity System
Target Users	Students, Faculty Members, Administrators
Problem Addressed	Copied, paraphrased, and AI-generated content
Solution Approach	Multi-layer NLP-based originality analysis
Deployment	Institutional server or cloud
Expected Outcome	Improved academic integrity

Table 2.1 CheckMate System Scope and Benefits

2.2 Project Functions

CheckMate performs several core functions essential for academic originality verification. The first major function **is** document upload and preprocessing, where submitted files are cleaned, tokenized, and prepared for analysis. This step ensures consistent input across different document formats.

The second function is plagiarism and similarity detection, which includes copied text detection using TF-IDF and cosine similarity, as well as paraphrase detection using semantic similarity models. These techniques identify both exact and reworded similarities.

Another critical function is AI-generated content detection, where transformer-based classifiers analyze writing patterns to estimate the likelihood of AI authorship. The system then computes a weighted originality score based on the outputs of all detection modules.

Finally, the platform generates detailed originality reports with highlighted sections, similarity percentages, and improvement suggestions. Faculty users can manage assignments and review submissions, while students can revise and resubmit work based on feedback

Function ID	Function Name	Description
F1	User Authentication	Secure login and role-based access
F2	Document Upload	PDF, DOCX, TXT upload
F3	Text Preprocessing	Cleaning and tokenization
F4	Copied Detection	TF-IDF + Cosine Similarity
F5	Paraphrase Detection	SBERT semantic similarity
F6	AI Detection	Transformer-based classifier
F7	Originality Score	Weighted score calculation
F8	Report Generation	Detailed originality report

Table 2.2 User Classes and Characteristics

2.3 User Classes and Characteristics

The system supports three primary user classes: Students, Faculty Members, and Administrators. Each user class interacts with the system differently and has distinct access privileges. Students are the most frequent users and primarily interact with document upload and report viewing features.

Faculty members use the system to create assignments, set similarity thresholds, review submissions, and provide feedback. They require detailed reports and visualization tools to evaluate originality efficiently. Faculty users also benefit from automated analysis, which reduces manual checking effort.

Administrators oversee system configuration, user management, and usage analytics. They ensure smooth operation, data security, and scalability. This role-based design ensures usability, security, and efficient workflow management across the platform.

2.4 Operating Environment

CheckMate operates in a web-based environment accessible through standard browsers such as Chrome and Edge. The frontend is developed using React.js, providing a responsive and user-friendly interface. Backend services are implemented using Node.js, handling authentication, file uploads, and report generation.

AI processing modules are implemented in Python, leveraging NLP and machine learning libraries. These modules can be deployed as independent services to improve scalability and performance. The system uses MongoDB for storing user data, documents, and analysis reports.

The platform can be deployed on institutional servers or cloud infrastructure, depending on scalability requirements. The operating environment is designed to support concurrent users while maintaining performance and reliability.

CHAPTER 3

REQUIREMENTS

3.1 Functional Requirements

The system must allow secure user registration and authentication for students and faculty. It must support document uploads in multiple formats, including PDF, DOCX, and TXT. Uploaded documents must be preprocessed and analyzed automatically without manual intervention.

The system must detect copied, paraphrased, and AI-generated content using appropriate algorithms. It must generate detailed originality reports highlighting problematic sections and providing similarity scores. Faculty users must be able to manage assignments, set similarity thresholds, and review submissions.

The system must store submission history and allow students to view previous reports. It should also support resubmission when permitted by faculty. These features ensure transparency and continuous improvement.

Req ID	Requirement
FR1	Secure login
FR2	Multi-format upload
FR3	Copied content detection
FR4	Paraphrase detection
FR5	AI content detection
FR6	Originality report generation

Table 3.1 Operating Environment Details

3.2 Non-Functional Requirements

The system must provide fast response times, even when handling multiple submissions simultaneously. Performance is critical to ensure timely feedback for students and faculty. The platform must be scalable to support increasing user numbers.

Security and privacy are essential nonfunctional requirements. User data and documents must be protected against unauthorized access. The system must also ensure reliability and minimal downtime.

Usability is another key requirement. The interface must be intuitive and accessible to users with varying technical expertise. Maintainability and extensibility are also important to support future enhancements.

Attribute	Description
Performance	Fast response time
Scalability	Supports multiple users
Security	Secure data & authentication
Reliability	Minimal downtime
Usability	Easy to use interface
Maintainability	Modular and extensible

Table 3.2 Non-Functional Requirements

3.3 Hardware Requirements

The system requires a server or cloud infrastructure with sufficient processing power to handle AI model execution. A minimum of 8 GB RAM and multi-core processor is recommended. Adequate storage is required for document uploads and reports.

Component	Specification
Processor	Multi-core CPU
RAM	Minimum 8 GB
Storage	100 GB or more
Client	Laptop/Desktop
Internet	Stable connection

Table 3.3 Hardware Requirements

3.4 Software Requirements

Software requirements include Python for AI processing, Node.js for backend services, React.js for frontend development, and MongoDB for database management. Additional libraries for NLP and machine learning are required.

Software	Purpose
Python	AI & NLP processing
Node.js	Backend services
React.js	Frontend UI
MongoDB	Database
NLP Libraries	Text analysis
Browser	Chrome / Edge

Table 3.4 Software Requirements

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The CheckMate system follows a modular and layered architecture. The frontend layer handles user interaction and visualization. The backend layer manages authentication, file handling, and communication between components.

The AI processing layer consists of independent modules for copied detection, paraphrase detection, and AI-generated content detection. Each module produces a score that contributes to the final originality calculation. This separation of concerns improves maintainability and scalability.

Data flows from document upload to preprocessing, analysis, scoring, and report generation. The modular design allows future integration of additional detection techniques without major system redesign.



Fig. 4.1 Technologies Used

Layer	Component	Description
Presentation	React UI	User interface
Application	Node.js	API & logic
Processing	AI Models	Plagiarism & AI detection
Data	MongoDB	Data storage

Table 4.1 Technology Integration Details

4.2 Data Flow Diagram (DFD)

The Data Flow Diagram represents the movement of data within the system. User-uploaded documents are passed to the preprocessing module for tokenization and cleaning. Processed data is then analyzed by plagiarism and AI-detection modules. Results are stored in the database and displayed to users through dashboards.

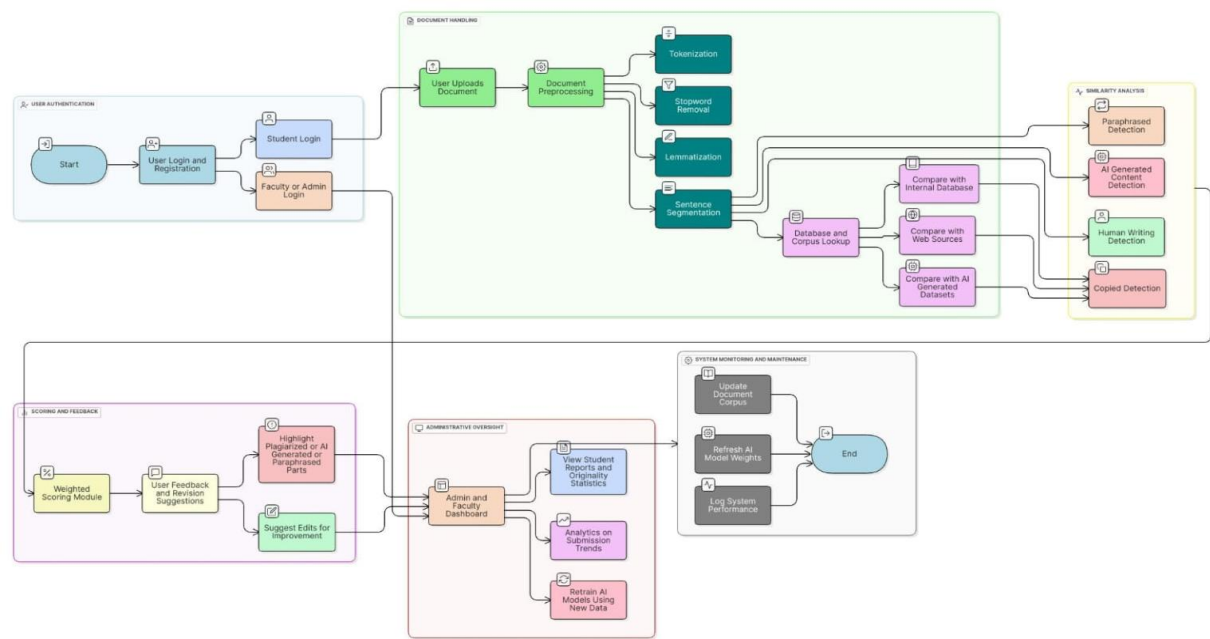


Fig. 4.2 System Methodology (Flow Diagram)

4.3 Module Description

The User Management module handles authentication and authorization. The Document Processing module extracts and preprocesses text. Similarity Detection module applies TF-IDF and semantic similarity algorithms. AI Detection module identifies AI-generated content. The Report Generation module compiles results into a detailed report.

4.4 AI Model Integration

AI Feature	Model / Dataset	Architecture Details	Key Output / Metric
Copied Content Detection	TF-IDF / Academic Text Corpus	Bag-of-words vectorization with term weighting using TF-IDF. Cosine similarity applied on document vectors for similarity computation.	Similarity Percentage indicating direct textual overlap.
Paraphrase Detection	SBERT / Pre-trained Sentence Transformer	Transformer-based Siamese architecture producing dense sentence embeddings. Semantic similarity computed using cosine similarity between embeddings.	Semantic Similarity Score highlighting paraphrased content.
AI-Generated Content Detection	Transformer Classifier / AI & Human Text Corpus	Fine-tuned transformer model analyzing linguistic patterns, coherence, and probability distributions.	AI Probability Score indicating likelihood of AI-generated text.
Originality Scoring Engine	Weighted Scoring Logic / System Output	Rule-based weighted aggregation of copied, paraphrased, and AI-detection outputs with configurable thresholds.	Final Percentage and Originality submission status (Pass/Flag).

Table 4.4 AI Model Integration Details

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Copied Content Detection Model Implementation Details

The copied content detection module was implemented using Python with the scikit-learn library. This feature utilizes the Term Frequency–Inverse Document Frequency (TF-IDF) vectorization technique combined with cosine similarity to identify direct textual overlap between documents. TF-IDF transforms textual data into weighted numerical vectors, allowing effective comparison of term importance across documents.

During implementation, documents are first preprocessed through tokenization, lowercasing, stop-word removal, and normalization. The TF-IDF vectorizer is trained on a reference corpus consisting of academic documents. Cosine similarity is then computed between the submitted document and stored references to measure textual similarity.

Performance Details:

- Similarity Threshold: 30% (configurable by faculty)
- Response Time: < 2 seconds for average-length documents
- Accuracy: High precision for detecting exact and near-exact copies

Term Frequency (TF):

$$TF(t, \text{text}) = \frac{\text{Number of times term } t \text{ appears in text}}{\text{Total number of terms in the text}}$$

Inverse Document Frequency (IDF):

$$IDF(t, \text{corpus}) = \log \left(\frac{\text{Total number of texts in corpus } |\text{corpus}|}{\text{Number of texts containing term } t} \right)$$

TF-IDF:

$$TF\text{-}IDF(t, \text{text}, \text{corpus}) = TF(t, \text{text}) \times IDF(t, \text{corpus})$$

Fig. 5.1 Copied Content Detection (TF-IDF Model)

5.2 Paraphrase Detection Model Implementation

The paraphrase detection module employs the Sentence-BERT (SBERT) model for semantic similarity analysis. This model captures contextual meaning beyond surface-level text matching and is capable of identifying paraphrased content where sentence structure and vocabulary have been altered while preserving the original meaning.

The implementation involves segmenting documents into sentences and encoding them into dense vector representations using SBERT. Pairwise cosine similarity is computed between sentence embeddings to detect semantic overlap. This approach significantly improves detection accuracy compared to traditional keyword-based methods.

Implementation Characteristics:

- Model Type: Pre-trained SBERT Transformer
- Similarity Metric: Cosine similarity on sentence embeddings
- Effective for: Rewritten and rephrased academic content

5.3 AI-Generated Content Detection Implementation

To address the increasing use of generative AI tools, CheckMate integrates a transformer-based AI content detection model. This model analyzes linguistic patterns, token probability distributions, and coherence features to differentiate between human-written and AI-generated text.

The model processes document segments and assigns an AI-likelihood score, indicating the probability that content was generated using AI-based writing tools. The detection logic is deployed as a Python service and exposed through an API for seamless integration with the backend.

Detection Characteristics:

- Output: AI probability score per document
- Inference Time: Near real-time processing
- Application: Academic assignments and research papers

5.4 Originality Scoring Engine Implementation

The originality scoring engine combines the outputs of the copied content detection, paraphrase detection, and AI-generated content detection modules. Each module contributes a weighted score to compute a final Originality Percentage. The scoring logic ensures balanced evaluation by penalizing excessive similarity and AI usage while rewarding genuinely original content. Threshold values and weight distributions are configurable by faculty members.

Scoring Logic:

- Copied Content Weight: High
- Paraphrased Content Weight: Moderate
- AI Content Weight: Moderate

Case	Expected meaning
label = 1 AND cosine similarity is high	Correct — model detects plagiarism / similarity
label = 1 BUT cosine similarity is low	Wrong — model failing to detect similarity
label = 0 AND cosine similarity is low	Correct — model detects no plagiarism
label = 0 BUT cosine similarity is high	Wrong — maybe false positive

Table 5.4 Interpretation of Similarity Detection Outcomes

5.5 Application Features Implemented

The user-facing features implemented in CheckMate include:

- Student Dashboard: Allows document upload, originality report viewing, and submission history tracking (see Fig. 5.5).
- Faculty Dashboard: Enables assignment creation, threshold configuration, and detailed report analysis (see Fig. 5.6).
- Document Management: Supports multiple file formats with automated preprocessing and storage.
- Report Generation: Generates downloadable originality reports with highlighted sections and scores.
- Scalability Framework: Designed to support institutional deployment and future LMS integration.

The web application interface and dashboard features are illustrated in Fig. 5.5 and Fig. 5.6, respectively.

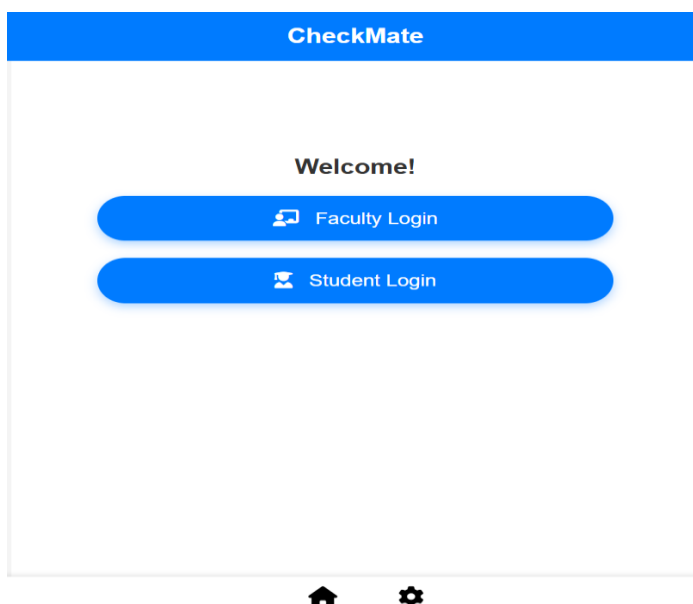


Fig. 5.5.i Student Dashboard Interface

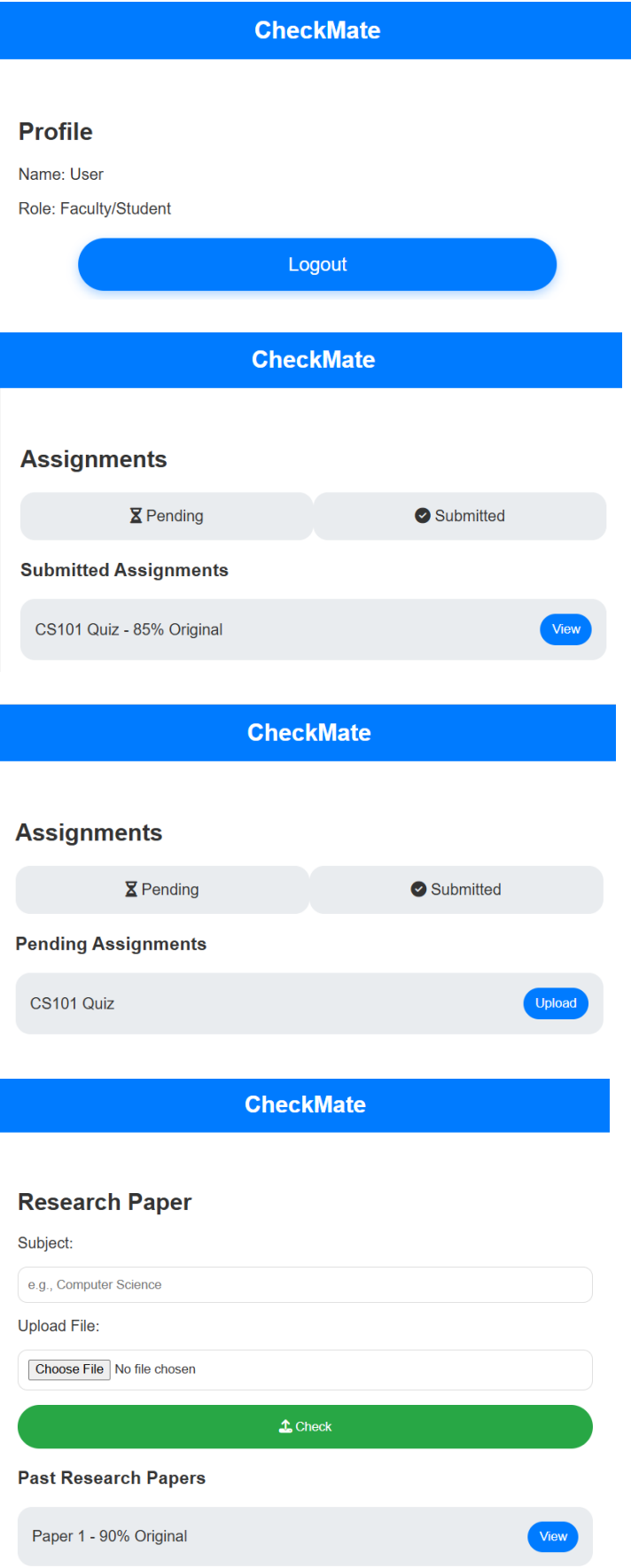
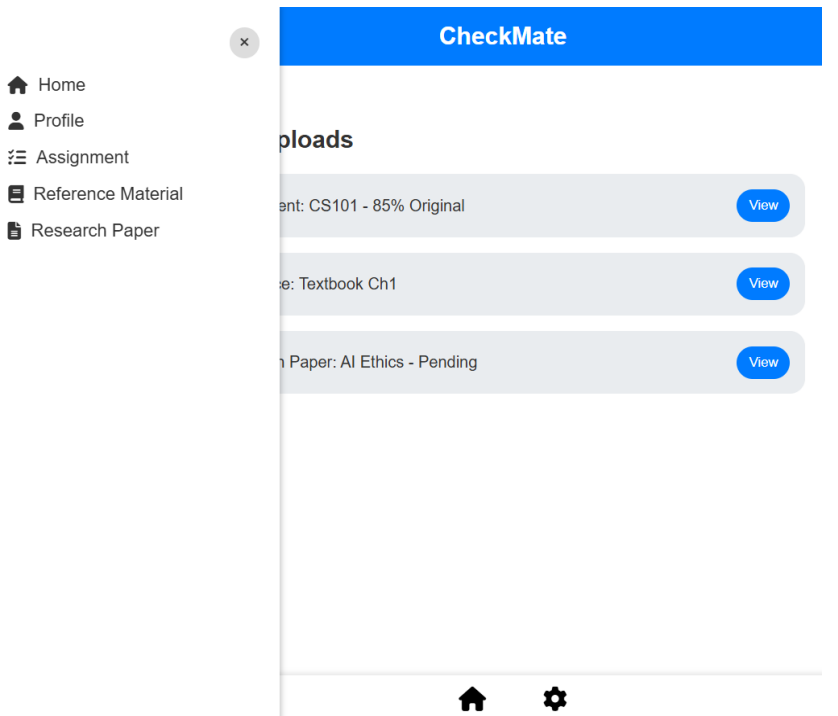


Fig. 5.5.ii Faculty Dashboard Interface



CheckMate

Assignment

Applicable Year:

Select Year

Subject:

e.g., Computer Science

Upload File:

Choose File No file chosen

Similarity Threshold (%):

15

Check

CheckMate

Reference Material

Subject:

e.g., Computer Science

Upload File:

Choose File No file chosen

Upload

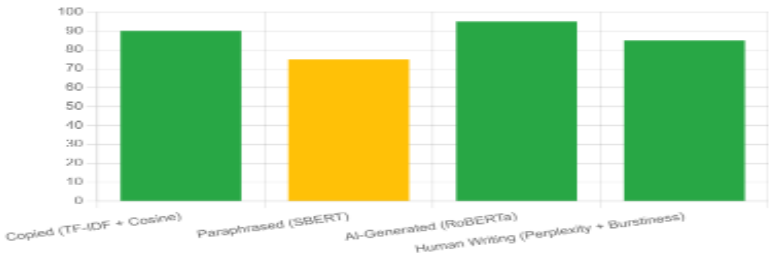
CheckMate Results

Research Paper Analysis

This is a dummy research paper paragraph for demonstration. Plagiarism is a rising issue in academics as increasing resources constantly, and it plays a high impact on student's performance and quality of education. Several studies have been carried out to mitigate the issue. This study provides a comprehensive analysis of the previous research, plagiarism detection approaches, existing plagiarism detection tools, type of the tools, features of popular plagiarism detection tools and the challenges in plagiarism detection. Plagiarism detection can be divided into source code and natural language plagiarism detection. Natural language plagiarism detection tools can be categorised based on the mode of detection, type of application, mode of services and languages.

AI Feedback: The highlighted sections show potential paraphrasing from known sources. Overall, the content is coherent but could benefit from more original phrasing.

Algorithm Scores



Scale: 0-100% (Higher = Better Originality; Green = Good, Red = Bad)



Copied Content: Low similarity detected in direct matches.



Paraphrased Sections: Sections 2 and 4 show semantic overlap – rephrase for originality.



AI-Generated: Minimal patterns; seems human-written.



Human Writing: High variation in style – authentic feel.



Other Feedback: Add more citations to avoid self-plagiarism risks.



Research Paper Approved (Overall Score: 85%)



Download Full Report

CONCLUSION

The CheckMate – AI Powered Plagiarism & Idea Originality Checker represents a significant advancement in maintaining academic integrity by integrating a user-friendly web platform with a robust, multi-layered Artificial Intelligence and Natural Language Processing framework. The project effectively addresses critical real-world challenges in academic evaluation, such as direct copying, sophisticated paraphrasing, and the rising misuse of AI-generated content—limitations that traditional plagiarism detection systems often fail to overcome.

Key Achievements and Validation: The technical effectiveness of CheckMate is validated through the successful integration and performance of multiple detection modules. The TF-IDF with cosine similarity model demonstrated high precision in identifying exact and near-exact textual overlaps. The SBERT-based semantic similarity model significantly improved the detection of paraphrased content by capturing contextual meaning beyond surface-level matching. Additionally, the transformer-based AI content detection model reliably identified AI-generated text by analyzing linguistic patterns and probability distributions. The originality scoring engine effectively combined these outputs to generate a clear, interpretable originality percentage, supporting fair and transparent evaluation.

Impact and Scalability: By providing role-based dashboards for students and faculty, CheckMate enhances transparency, accountability, and efficiency in academic assessment. The system reduces faculty workload while encouraging students to improve originality through actionable feedback. The modular architecture, built using React.js, Node.js, Python, and MongoDB, ensures scalability and adaptability for institutional deployment across departments or universities.

Future Directions: Future enhancements will focus on improving detection robustness through fine-tuned domain-specific models, deeper explainability of AI decisions, and tighter integration with Learning Management Systems (LMS). Additional analytics for academic trend analysis and policy enforcement can further strengthen institutional governance. Overall, CheckMate stands as a complete, scalable, and practical solution poised to significantly improve academic integrity in modern educational environments.

REFERENCES

- [1] S. Meyer zu Eissen and B. Stein, "Intrinsic Plagiarism Detection," in *Proceedings of the European Conference on Information Retrieval*, 2006.
- [2] C. Potthast, B. Stein, and M. Anderka, "A Wikipedia-Based Multilingual Retrieval Model," *Information Retrieval Journal*, 2010.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP*, 2019.
- [5] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, 1988.
- [6] T. Mikolov et al., "Distributed Representations of Words and Phrases and their Compositionality," in *Proceedings of NeurIPS*, 2013.
- [7] OpenAI, "GPT Models and AI-Generated Text Characteristics," Technical Report, 2023.
- [8] A. Vaswani et al., "Attention Is All You Need," in *Proceedings of NeurIPS*, 2017.
- [9] Scikit-learn Developers, "Machine Learning in Python," *Journal of Machine Learning Research*, 2011.
- [10] MongoDB Inc., "MongoDB Documentation," 2024.
- [11] React Developers, "React: A JavaScript Library for Building User Interfaces," Meta Platforms, 2024.
- [12] Node.js Foundation, "Node.js Documentation," 2024.